

Protocolo individual

Nombre: Esteban David Marrugo Jassir

Programa: Ingeniería de Software

Asignatura: Algoritmos y Programación Básica

Docente: Heybertt Moreno Díaz

Descripción del texto o actividad a realizar:

Revisión del módulo de la Unidad 2 sobre estructuras de control, articulando los conceptos teóricos con mi experiencia práctica en desarrollo de software: tanto en el proyecto formativo NOEST del SENA ADSO (aplicación de escritorio en Java) como en Jules, mi proyecto personal en Python (asistente virtual). Analicé cómo el manejo de la secuencia, la selección condicional (if/else/switch) y los ciclos (for/while/do-while) determinan la estabilidad, la legibilidad y la prevención de errores en tiempo de ejecución.

Palabras claves:

Estructuras de control, flujo de ejecución, selección condicional, evaluación booleana, cortocircuito lógico, ciclos iterativos, ámbito de variables (scope), proyecto NOEST, proyecto Jules.

Objetivos de las lecturas o actividad a realizar:

- Comprender cómo las estructuras de control dirigen el flujo de un programa y por qué son la base para que un algoritmo resuelva problemas de forma estructurada.
- Contrastar la rigidez del tipado estático de Java en NOEST con la flexibilidad dinámica de Python en Jules.
- Analizar el cortocircuito lógico (&&, ||) como mecanismo de programación defensiva frente a excepciones de puntero nulo (NullPointerException).
- Comparar el ámbito de las variables (scope): el alcance de bloque en Java frente al alcance de función en Python.
- Establecer criterios claros para elegir entre for, while y do-while según la naturaleza de la tarea y el control del estado.

Conceptos claves y definiciones:

- **Estructuras de control:** Instrucciones que alteran la ejecución lineal del procesador para decidir qué bloque se ejecuta y cuándo, según condiciones definidas por el programador.
- **Secuencia:** Flujo por defecto en el que las instrucciones se ejecutan una tras otra, en el orden exacto en que fueron escritas.
- **Selección condicional (if, else if, else):** Mecanismo de bifurcación que evalúa una condición booleana y ejecuta un camino si se cumple o salta a otro si no.

- **Selección múltiple (switch-case):** Estructura que evalúa una variable contra varios valores constantes discretos. En Java requiere la sentencia break al final de cada caso para evitar la ejecución no deseada de los bloques siguientes.
- **Ciclos o bucles (for, while, do-while):** Bloques que repiten una tarea mientras una condición lógica se mantenga verdadera. Requieren una condición de salida o actualización de la variable de control para no caer en bucles infinitos.
- **Pre-test (while, for) frente a post-test (do-while):** En el pre-test la condición se evalúa antes de entrar al bloque (puede no ejecutarse ninguna vez); en el post-test el bloque se ejecuta al menos una vez antes de verificar la condición.
- **Ámbito de variable (scope):** Región del código donde una variable es visible y válida. En Java, una variable declarada dentro de un bloque {} nace y muere en él; en Python el alcance es por función, por lo que una variable creada en un for sigue existiendo fuera del bucle.

Resumen de la(as) lecturas:

Al revisar el módulo de estructuras de control, los conceptos no me resultaron raros. En el SENA vengo trabajando en el proyecto formativo NOEST, donde construimos una aplicación de escritorio en Java, y de forma independiente desarrollo Jules, un proyecto propio en Python enfocado en automatización y modelos de lenguaje. Para mí, lo más enriquecedor de esta unidad no fue memorizar la sintaxis de cada sentencia, sino reflexionar sobre cómo cada lenguaje responde cuando ponemos a prueba la lógica de negocio.

El primer contraste fuerte lo encontré en la selección condicional. Python maneja una evaluación implícita de veracidad (truthiness): en Jules, si escribo if respuesta:, el flujo asume que es falso si la variable llega como una lista vacía [], un diccionario vacío {} o el número 0. Esto resulta cómodo y ahorra líneas, pero puede ocultar fallas lógicas cuando una lista vacía sí representa un resultado válido de la consulta. En Java, como lo experimento en NOEST, el compilador es mucho más estricto, la condición del if exige un tipo boolean explícito. Si escribo if (respuesta), el código no compila.

Aquí es donde el cortocircuito lógico (&&) se convierte en una herramienta de programación defensiva esencial. Si respuesta es nula, el compilador no evalúa el segundo término, lo que evita que el programa lance un NullPointerException y bloquee la aplicación. Por esta razón, el orden de los factores sí altera el producto: la validación de nulidad siempre debe anteceder al llamado del método.

Con la selección múltiple (switch-case) ocurre algo similar, en la interfaz de NOEST, para gestionar las opciones de un menú o los estados de un registro, encadenar cinco o seis else if vuelve el código denso y propenso a errores. El switch organiza las ramas de forma limpia al contrastar valores discretos como enteros, caracteres, cadenas o enumeraciones. Su aspecto crítico es el manejo del break: omitirlo provoca el efecto de cascada (fall-through), haciendo que el programa ejecute los bloques siguientes sin verificar sus

condiciones. Es un error silencioso porque compila sin advertencias, pero altera completamente el comportamiento esperado.

Otro punto que me pareció muy formativo fue el ámbito de las variables (scope). En Java, si declaro una variable en la cabecera de un ciclo como `for (int i = 0; i < total; i++)`, su ciclo de vida queda limitado exclusivamente a las llaves `{}` del bucle; cualquier intento de usar `i` afuera genera un error de compilación. En Python, en cambio, la variable que itera en `for item in coleccion`: sigue disponible en toda la función después de terminar el ciclo. Si más adelante en el archivo vuelvo a usar `item` pensando que es una variable nueva, puedo arrastrar valores residuales difíciles de depurar. Entender este detalle me ayuda a escribir código más limpio y modular.

Por último, respecto a las estructuras de repetición, la lectura aporta un criterio técnico muy claro. Cuando conozco con certeza cuántas veces debo procesar un dato (como iterar una tabla de usuarios en NOEST), la elección natural es un `for`. Pero cuando la repetición depende de un evento que no controlo en el tiempo (la respuesta de un socket o la confirmación del usuario en Jules), el ciclo `while` es el indicado. El `do-while` resuelve con elegancia situaciones donde necesito presentar un menú o pedir una entrada al menos una primera vez antes de validar el dato. Como lo dicen Ramírez (2007) y Joyanes Aguilar (2020), el cuidado principal en cualquier ciclo no controlado por contador está en actualizar oportunamente la variable de control; descuidar esa condición produce un bucle infinito que consume recursos y congela el proceso principal.

Metodología de trabajo:

Descargué el pdf de la guía desde el SIMA, lo leí y plasme las ideas en mi bóveda de Obsidian. Luego subí el documento a NotebookLM para generar preguntas de autoevaluación y escuchar el resumen en audio, lo que me ayuda a repasar los conceptos de forma dinámica. Para fijar el aprendizaje en la práctica, abrí mi terminal en Linux y ejecuté pruebas de código comparativas con mis proyectos NOEST y Jules: probé con `javac` y `python` cómo se comportan las condiciones con valores vacíos, los ciclos anidados y el cortocircuito, verificando de primera mano la diferencia entre el tipado estático y el dinámico.

Conclusiones de la lectura o actividad:

- Las estructuras de control son el núcleo de la programación estructurada: permiten que un algoritmo deje de ser una lista fija de instrucciones y adquiera la capacidad de tomar decisiones según el estado de los datos.
- Trabajar en paralelo con Java (NOEST) y Python (Jules) me demostró que la base algorítmica es universal; lo que cambia es el nivel de exigencia del compilador, que en Java obliga a una mayor disciplina defensiva desde la escritura del código.
- El cortocircuito lógico (`&&`, `||`) es una práctica fundamental de seguridad en el desarrollo: evita excepciones en tiempo de ejecución al validar referencias antes de invocar métodos.

- Comprender la diferencia entre el ámbito de bloque en Java y el ámbito de función en Python ayuda a prevenir la sobreescritura accidental de variables y facilita el mantenimiento del software.
- La elección de las estructuras repetitivas no es arbitraria: el ciclo for responde a iteraciones acotadas, mientras que while y do-while se ajustan a procesos dependientes de eventos o validaciones interactivas.

Discusiones y recomendaciones:

- La mejor forma de interiorizar estas estructuras es tirando código real y haciendo ejercicios o proyectos para poner en practica lo que se aprende en cada unidad.
- Los conceptos del módulo me quedaron claros y complementa de forma directa con lo que desarrollo a diario, por lo que no tengo dudas pendientes sobre la temática de esta unidad.

Bibliografía:

- Joyanes Aguilar, L. (2020). Fundamentos de programación: algoritmos, estructura de datos y objetos (5.ª ed.). McGraw-Hill.
- Ramírez, F. (2007). Introducción a la programación: Algoritmos y su implementación en VB.NET, C#, Java y C++. Alfaomega Grupo Editor.
- Trejos Buriticá, O. I. (2021). Lógica de programación: solucionario en pseudocódigo. Ediciones de la U.
- Universidad de Cartagena - CTEV. (2026). Módulo Didáctico Unidad 2: Estructuras de Control — Algoritmos y Programación Básica. Facultad de Ingeniería, Universidad de Cartagena.
- Valls Ferrán, J. M., & Camacho Fernández, D. (2003). Programación, algoritmos y ejercicios resueltos en JAVA. Pearson Educación.
- Wanumen Silva, L. F., et al. (2017). Java básico. Ecoe Ediciones.